Chapter 1 – The DUT

This training guide will focus on showing how we can build a basic UVM environment, so the device under test was kept very simple in order to emphasize the explanation of UVM itself.

The DUT used is a simple ALU, limited to a single operation: the add operation. The inputs and outputs are represented in Figure 1.1.

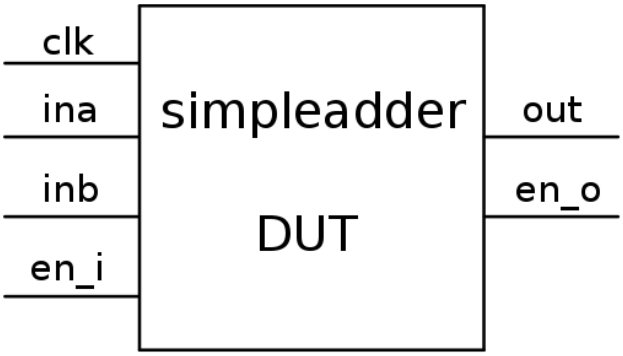


Figure 1.1: Representation of the DUT's inputs/outputs

This DUT takes two values of 2 bits each, *ina* and *inb*, sums them and sends the result to the output *out*. The inputs are sampled to the signal of *en_i* and the output is sent at the same time *en_o* is signalled.

The operation of the DUT is represented as a timing diagram and as a state machine in Figure 1.2.

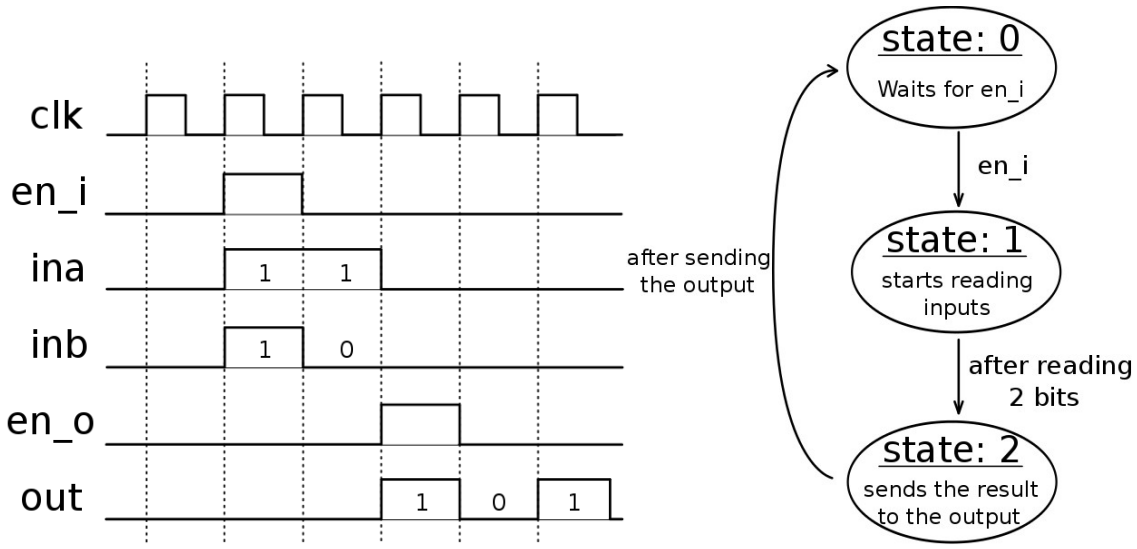


Figure 1.2: Operation of the DUT

The code for this DUT can be found here:

- simpleadder.v

Chapter 2 – Defining the verification environment

Before understanding UVM, we need to understand verification.

Right now, we have a DUT and we will have to interact with it in order to test its functionality, so we need to stimulate it. To achieve this, we will need a block that generates sequences of bits to be transmitted to the DUT, this block is going to be named *sequencer*.

Usually sequencers are unaware of the communication bus, they are responsible for generating generic sequences of data and they pass that data to another block that takes care of the communication with the DUT. This block will be the *driver*.

While the driver maintains activity with the DUT by feeding it data generated from the sequencers, it doesn't do any validation of the responses to the stimuli. We need another block that listens to the communication between the driver and the DUT and evaluates the responses from the DUT. This block is the *monitor*.

Monitors sample the inputs and the outputs of the DUT, they try to make a prediction of the expected result and send the prediction and result of the DUT to another block, the *scoreboard*, in order to be compared and evaluated.

All these blocks constitute a typical system used for verification and it's the same structure used for UVM testbenches.

You can find a representation of a similar environment in Figure 2.1.

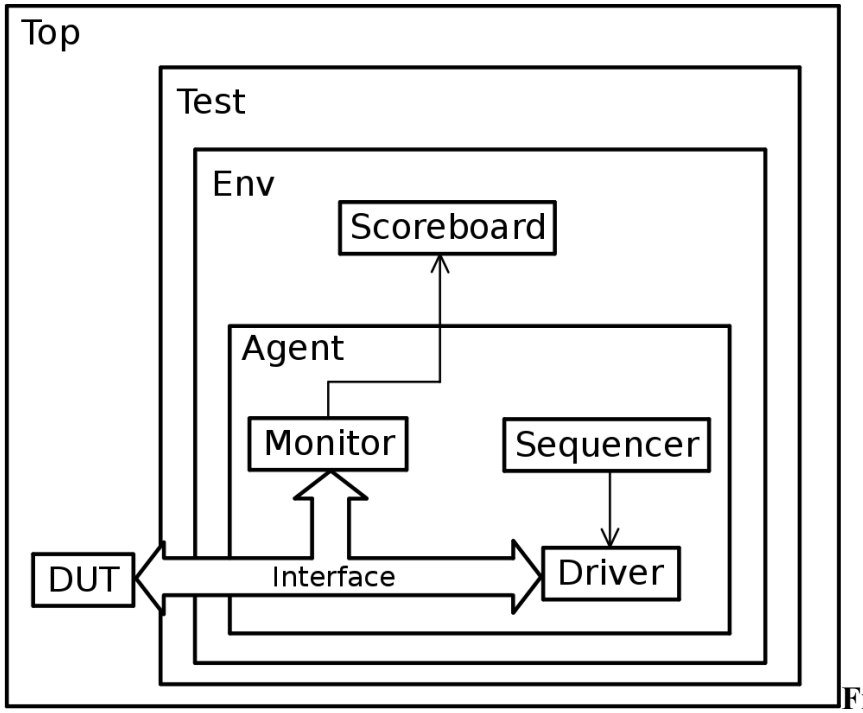


Figure 2.1: Typical UVM testbench

Usually, sequencers, drivers and monitors compose an *agent*. An agent and a scoreboard compose an *environment*. All these blocks are controlled by a greater block denominated of *test*. The *test* block controls all the blocks and sub blocks of the testbench. This means that just by changing a few lines of code, we could add, remove and override blocks in our testbench and build different environments without rewriting the whole test.

To illustrate the advantage of this feature, let's imagine a situation where we are testing a another DUT that uses SPI for communication. If, by any chance, we want to test a similar DUT but with I2C instead, we would just need to add a monitor and a driver for I2C and override the existing SPI blocks, the sequencer and the scoreboard could reused just fine.

UVM Classes

The previous example demonstrates one of the great advantages of UVM. It's very easy to replace components without having to modify the entire testbench, but it's also due to the concept of classes and objects from SystemVerilog.

In UVM, all the mentioned blocks are represented as objects that are derived from the already existent classes.

A class tree of the most important UVM classes can be seen in Figure 2.2.

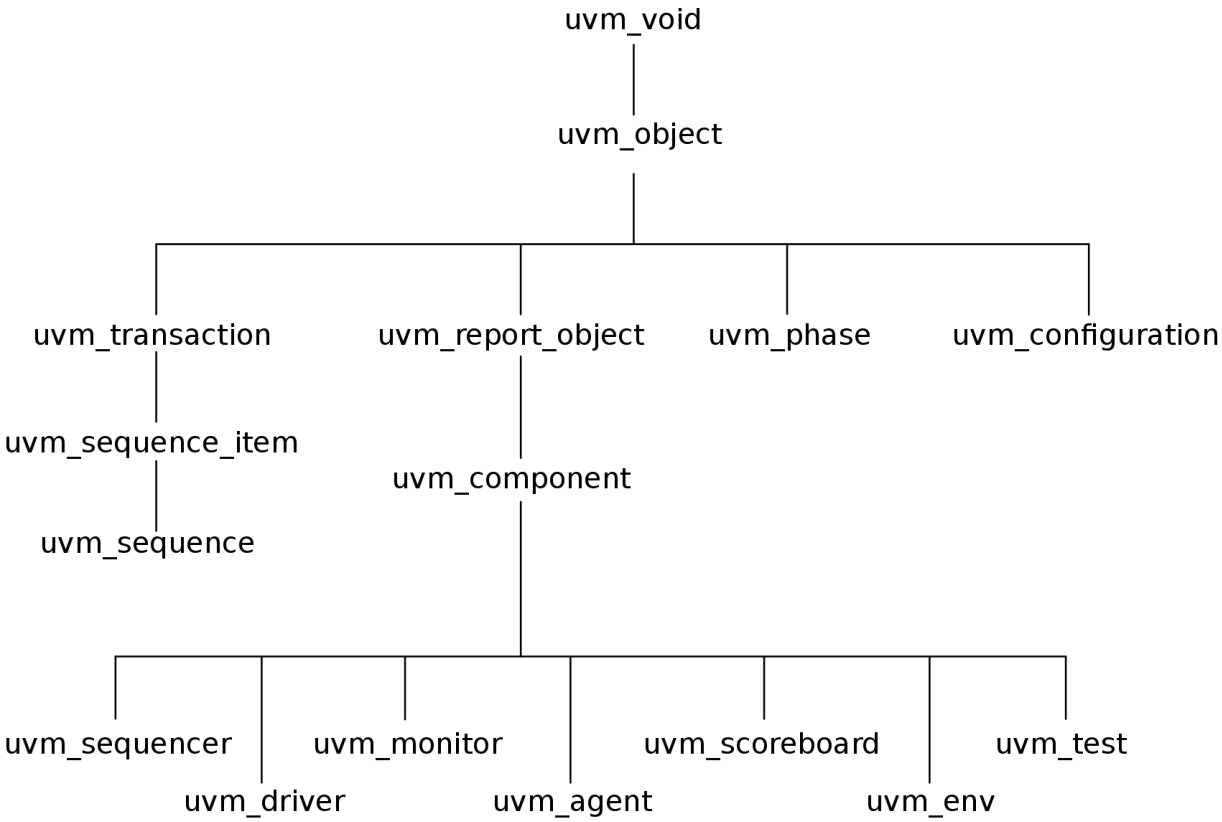


Figure 2.2: Partial UVM class tree

The data that travels to and from our DUT will stored in a class derived either from *uvm_sequence_item* or *uvm_sequence*. The sequencer will be derived from *uvm_sequencer*, the driver from *uvm_driver*, and so on.

Every each of these classes already have some useful methods implemented, so that the designer can only focus on the important part, which is the functional part of the class that will verify the design. These methods are going to addressed further ahead.

For more information about UVM classes, you can consult the document *Accellera's UVM 1.1 Class Reference*.

UVM Phases

All these classes have simulation phases. Phases are ordered steps of execution implemented as methods. When we derive a new class, the simulation of our testbench will go through these different steps in order to construct, configure and connect the testbench component hierarchy.

The most important phases are represented in Figure 2.3.

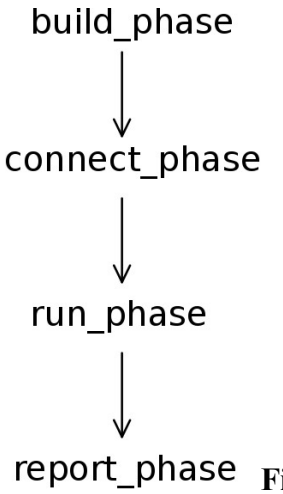


Figure 2.3: Partial list of UVM phases

A brief explanation of each phase will follow:

- The build phase is used to construct components of the hierarchy. For example, the build phase of the agent class will construct the classes for the monitor, for the sequencer and for the driver.

- The connect is used to connect the different sub components of a class. Using the same example, the connect phase of the agent would connect the driver to the sequencer and it would connect the monitor to an external port.

- The run phase is the main phase of the execution, this is where the actual code of a simulation will execute.

- And at last, the report phase is the phase used to display the results of the simulation.

There are many more phases but none of them are mandatory. If we don't need to have one in a particular class, we can just omit it and UVM will ignore it.

More information about UVM phasing can be consulted in Verification Academy's UVM Cookbook, page 48.

UVM Macros

Another important aspect of UVM are the macros. These macros implement some useful methods in classes and in variables. they are optional, but recommended.

The most common ones are:

- `uvm_component_utils – This macro registers the new class type. It's usually used when deriving new classes like a new agent, driver, monitor and so on.

- `uvm_field_int – This macro registers a variable in the UVM factory and implements some functions like copy(), compare() and print().

- `uvm_info – This a very useful macro to print messages from the UVM environment during simulation time.

This guide will not go into much detail about macros, their usage is always the same for every class, so it's not worth to put much thought into it for now.

More information can be found in *Accellera's UVM 1.1 Class Reference*, page 405.

Typical UVM class

All this said, a typical UVM class will look a lot like the one described in Code 2.1.

```
1         class generic_component extends uvm_component;
2        `uvm_component_utils(generic_component)
3
4        function new(string name, uvm_component parent);
5             super.new(name, parent);
6        endfunction: new
7
8        function void build_phase(uvm_phase phase);
9             super.build_phase(phase);
10
11            //Code for constructors goes here
12       end_function: build_phase
13
14       function void connect_phase(uvm_phase phase);
15            super.connect_phase(phase);
16
17            //Code for connecting components goes here
18       endfunction: connect_phase
19
20       task run_phase(uvm_phase phase);
21            //Code for simulation goes here
22       endtask: run_phase
23
24       function void report_phase(uvm_phase phase);
25            //Code for showing simulation results goes here
26       endfunction: report_phase
27  endclass: generic_component
```

Code 2.1: Code for a generic component

The code listed here, is the most basic sample that all components will share as you will see from now on.

SimpleAdder UVM Testbench

After a brief overview of a UVM testbench, it's time to start developing one. By the end of this guide, we will have the verification environment from the Figure 2.4.

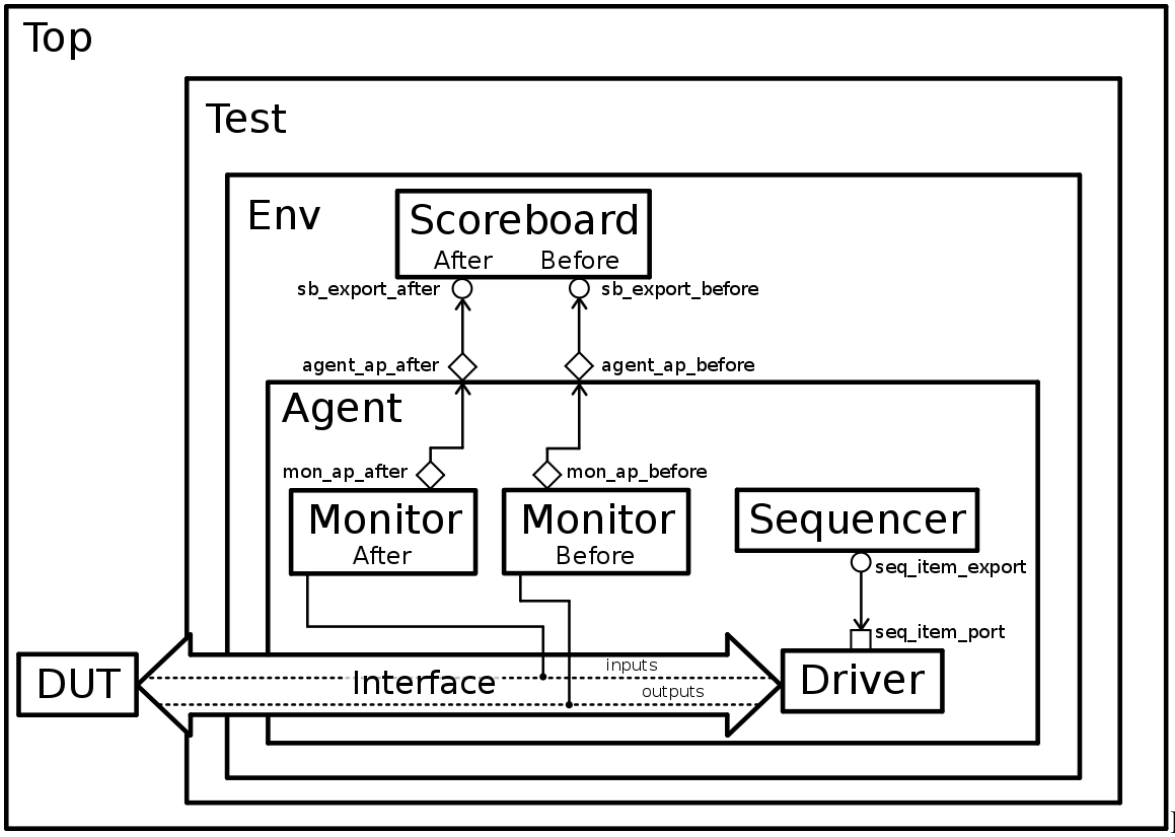


Figure 2.4: SimpleAdder Final Testbench

This guide will begin to approach the *top block* and the *interface* (chapter 3), then it will explain what data will be generated with the *sequences* and *sequencers* on chapter 4.

Following the sequencers, it will explain how to drive the signals into the DUT and how to observe the response in chapters 5 and 6 respectively.

Subsequently, it will explain how to connect the sequencer to the driver and the monitor to the scoreboard in chapter 7. Then it will show to build a simple *scoreboard* in chapter 8.

And finally, the test will be executed and analyzed.

The testbench can be run with the execution of a Makefile provided in the repository. As I mentioned previously, this Makefile uses Synopsys VCS but it should be easily modifiable to be executed with any HDL simulator.

Chapter 3 – Top Block

In a normal project, the development of the DUT is done separately from the development of the testbench, so there are two components that connects both of them:

- The top block of the testbench
- A virtual interface

The top block will create instances of the DUT and of the testbench and the virtual interface will act as a bridge between them.

The interface is a module that holds all the signals of the DUT. The monitor, the driver and the DUT are all going to be connected to this module.

The code for the interface can be seen in Code 3.1.

```
1   interface simpleadder_if;
2        logic    sig_clock;
3        logic    sig_ina;
4        logic    sig_inb;
5        logic    sig_en_i;
6        logic    sig_out;
7        logic    sig_en_o;
8   endinterface: simpleadder_if
```

Code 3.1: Interface module – *simpleadder_if.sv*

After we have an interface, we will need the top block. This block will be a normal SystemVerilog module and it will be responsible for:

- Connecting the DUT to the test class, using the interface defined before.
- Generating the clock for the DUT.
- Registering the interface in the UVM factory. This is necessary in order to pass this interface to all other classes that will be instantiated in the testbench. It will be registered in the UVM factory by using the *uvm_resource_db* method and every block that will use the same interface, will need to get it by calling the same method. It might start to look complex, but for now we won't need to worry about it too much.
- Running the test.

The source for the top block is represented in Code 3.2.

```
1  `include "simpleadder_pkg.sv"
2  `include "simpleadder.v"
3  `include "simpleadder_if.sv"
4
5  module simpleadder_tb_top;
6       import uvm_pkg::*;
7
8       //Interface declaration
9       simpleadder_if vif();
10
11      //Connects the Interface to the DUT
12      simpleadder dut(vif.sig_clock,
13                      vif.sig_en_i,
14                      vif.sig_ina,
15                      vif.sig_inb,
16                      vif.sig_en_o,
17                      vif.sig_out);
18      initial begin
```

```
19            //Registers the Interface in the configuration block
20            //so that other blocks can use it
21            uvm_resource_db#(virtual
22  simpleadder_if)::set(.scope("ifs"), .name("simpleadder_if"), .val(vif));
23
24            //Executes the test
25            run_test();
26      end
27
28      //Variable initialization
29      initial begin
30            vif.sig_clock = 1'b1;
31      end
32
33      //Clock generation
34      always
35            #5 vif.sig_clock = ~vif.sig_clock;
        endmodule
```

Code 3.2: Top block – *simepladder_tb_top.sv*

A brief explanation of the code will follow:

- The lines 2 and 3 include the DUT and the interface into the top block, the line 5 imports the UVM library, lines 11 to 16 connect the interface signals to the DUT.
- Line 21 registers the interface in the factory database with the name *simpleadder_if*.
- Line 24 runs one of the test classes defined at compilation runtime. This name is specified in the Makefile.
- Line 34 generates the clock with a period of 10 timeunits. The timeunit is also defined in the Makefile.

For more information about interfaces, you can consult the book "SystemVerilog for Verification: A Guide to Learning the TestBench Language Features", chapter 5.3.

Chapter 4 – Sequences and sequencers

The first step in verifying a RTL design is defining what kind of data should be sent to the DUT. While the driver deals with signal activities at the bit level, it doesn't make sense to keep this level of abstraction as we move away from the DUT, so the concept of transaction was created.

A transaction is a class object, usually extended from *uvm_transaction* or *uvm_sequence_item* classes, which includes the information needed to model the communication between two or more components.

Transactions are the smallest data transfers that can be executed in a verification model. They can include variables, constraints and even methods for operating on themselves. Due to their high abstraction level, they aren't aware of the communication protocol between the

components, so they can be reused and extended for different kind of tests if correctly programmed.

An example of a transaction could be an object that would model the communication bus of a master-slave topology. It could include two variables: the address of the device and the data to be transmitted to that device. The transaction would randomize these two variables and the verification environment would make sure that the variables would assume all possible and valid values to cover all combinations.

In order to drive a stimulus into the DUT, a driver component converts transactions into pin wiggles, while a monitor component performs the reverse operation, converting pin wiggles into transactions.

After a basic transaction has been specified, the verification environment will need to generate a collection of them and get them ready to be sent to the driver. This is a job for the *sequence*. Sequences are an ordered collection of transactions, they shape transactions to our needs and generate as many as we want. This means if we want to test just a specific set of addresses in a master-slave communication topology, we could restrict the randomization to that set of values instead of wasting simulation time in invalid values.

Sequences are extended from *uvm_sequence* and their main job is generating multiple transactions. After generating those transactions, there is another class that takes them to the driver: the sequencer. The code for the sequencer is usually very simple and in simple environments, the default class from UVM is enough to cover most of the cases.

A representation of this operation is shown in Figure 4.1.

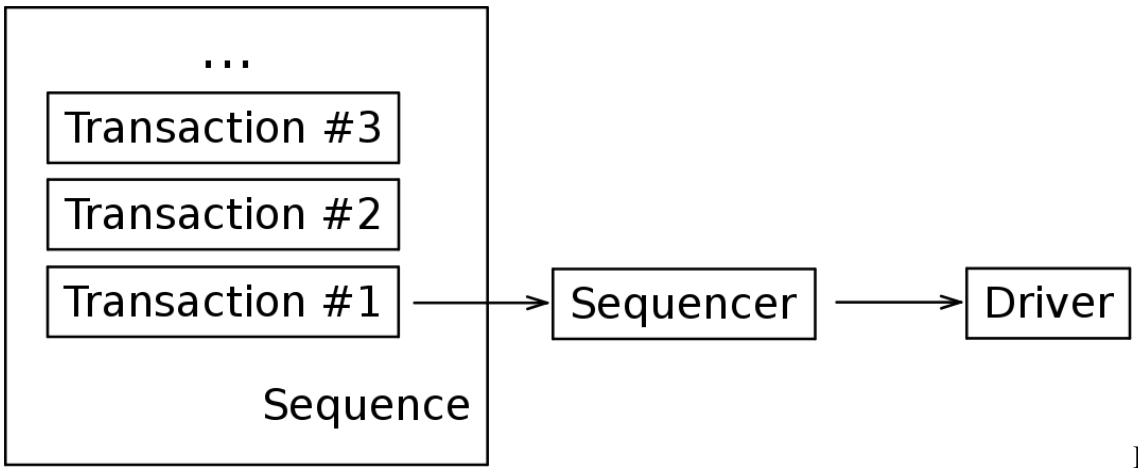


Figure 4.1 – Relation between a sequence, a sequencer and a driver

The sequence englobes a group of transactions and the sequencer takes a transaction from the sequence and takes it to the driver.

To test our DUT we are going to define a simple transaction, extended from *uvm_sequence_item*. It will include the following variables:

- rand bit[1:0] ina

- rand bit[1:0] inb
- bit[2:0] out

The variables *ina* and *inb* are going to be random values to be driven to the inputs of the DUT and the variable *out* is going to store the result. The code for the transaction is represented in Code 4.1.

```
1  class simpleadder_transaction extends uvm_sequence_item;
2       rand bit[1:0] ina;
3       rand bit[1:0] inb;
4       bit[2:0] out;
5
6       function new(string name = "");
7            super.new(name);
8       endfunction: new
9
10      `uvm_object_utils_begin(simpleadder_transaction)
11      `uvm_field_int(ina, UVM_ALL_ON)
12      `uvm_field_int(inb, UVM_ALL_ON)
13      `uvm_field_int(out, UVM_ALL_ON)
14      `uvm_object_utils_end
15 endclass: simpleadder_transaction
```

Code 4.1 – Transaction for the simpleadder

An explanation of the code will follow:

- Lines 2 and 3 declare the variables for both inputs. The *rand* keyword asks the compiler to generate and store random values in these variables.
- Lines 6 to 8 include the typical class constructor.
- Lines 10 to 14 include the typical UVM macros.

These few lines of code define the information that is going to be exchanged between the DUT and the testbench.

To demonstrate the reuse capabilities of UVM, let's imagine a situation where we would want to test a similar adder with a third input, a port named *inc*.

Instead of rewriting a different transaction to include a variable for this port, it would be easier just to extend the previous class to support the new input.

It's possible to see an example in Code 5.2.

```
1  class simpleadder_transaction_3inputs extends simpleadder_transaction;
2       rand bit[1:0] inc;
3
4       function new(string name = "");
5            super.new(name);
6       endfunction: new
7
```

```
8       `uvm_object_utils_begin(simpleadder_transaction_3inputs)
9       `uvm_field_int(inc, UVM_ALL_ON)
10      `uvm_object_utils_end
11 endclass: simpleadder_transaction_3inputs
```

Code 5.2 – Extension of the previous transaction

As a result of the class *simpleadder_transaction_3inputs* being an extension of *simpleadder_transaction*, we didn't need to declare again the other variables. While in small examples, like this one, this might not look like something useful, for bigger verification environments, it might save a lot of work.

Sequence

Now that we have a transaction, the next step is to create a sequence.

The code for the sequencer can be found in Code 5.3.

```
1  class simpleadder_sequence extends
2  uvm_sequence#(simpleadder_transaction);
3       `uvm_object_utils(simpleadder_sequence)
4
5       function new(string name = "");
6            super.new(name);
7       endfunction: new
8
9       task body();
10           simpleadder_transaction sa_tx;
11
12           repeat(15) begin
13                sa_tx = simpleadder_transaction::type_id::create(...
14
15                start_item(sa_tx);
16                     assert(sa_tx.randomize());
17                finish_item(sa_tx);
18           end
19      endtask: body
   endclass: simpleadder_sequence
```

Code 5.3 – Code for the sequencer

An explanation of the code will follow:

- Line 8 starts the task *body()*, which is the main task of a sequence
- Line 11 starts a cycle in order to generate 15 transactions
- Line 12 initializes a blank transaction
- Line 14 is a call that blocks until the driver accesses the transaction being created
- Line 15 triggers the *rand* keyword of the transaction and randomizes the variables of the transaction to be sent to the driver

- Line 16 is another blocking call which blocks until the driver has completed the operation for the current transaction

Sequencer

The only thing missing is the sequencer. The sequence will be extended from the class *uvm_sequencer* and it will be responsible for sending the sequences to the driver. The sequencer gets extended from *uvm_sequencer*. The code can be seen on Code 5.4.

```
1 typedef uvm_sequencer#(simpleadder_transaction) simpleadder_sequencer;
```

Code 5.4 – Extension of the previous transaction

The code for the sequencer is very simple, this line will tell UVM to create a basic sequencer with the default API because we don't need to add anything else.

So, right now our environment has the following structure:

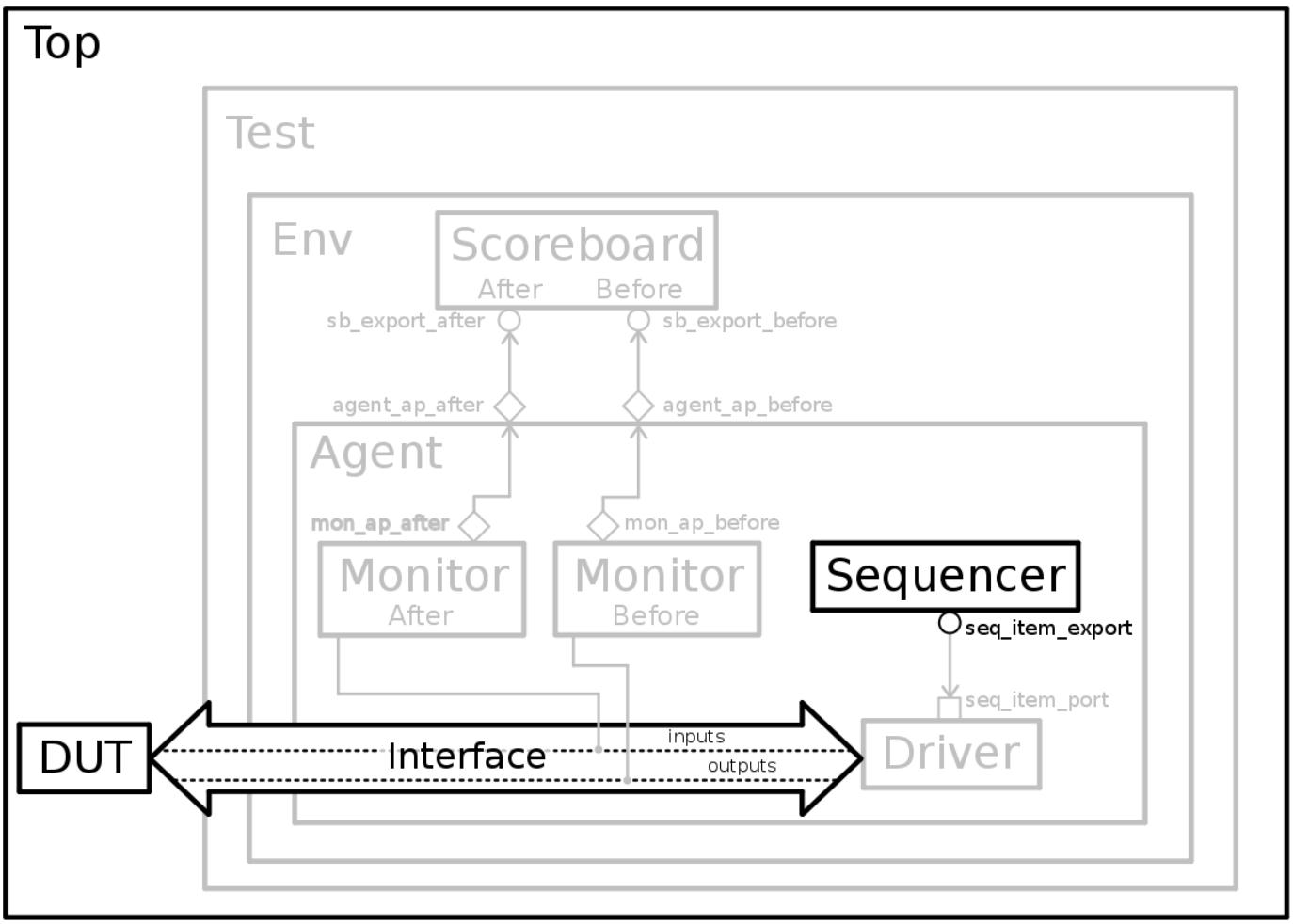


Figure 4.2 – State of the verification environment after the sequencer

You might have noticed two things missing:

- How does the sequence connects to the sequencer?
- How does the sequencer connects to the driver

The connection between the sequence and the sequencer is made by the *test* block, we will come to this later on chapter 10, and the connection between the sequencer and the driver will be explained on chapter 7.

For more information about transactions and sequences, you can consult:

- Accellera's UVM 1.1 User's Guide, page 48
- Verification Academy's UVM Cookbook, pages 188 and 200

Chapter 5 – Driver

The driver is a block whose role is to interact with the DUT. The driver pulls transactions from the sequencer and sends them repetitively to the signal-level interface. This interaction will be observed and evaluated by another block, the monitor, and as a result, the driver's functionality should only be limited to send the necessary data to the DUT.

In order to interact with our adder, the driver will execute the following operations: control the *en_i* signal, send the transactions pulled from the sequencer to the DUT inputs and wait for the adder to finish the operation.

So, we are going to follow these steps:

1. Derive the driver class from the *uvm_driver* base class
2. Connect the driver to the signal interface
3. Get the item data from the sequencer, drive it to the interface and wait for the DUT execution
4. Add UVM macros

In Code 5.1 you can find the base code pattern which is going to be used in our driver.

```
1  class simpleadder_driver extends uvm_driver#(simpleadder_transaction);
2        `uvm_component_utils(simpleadder_driver)
3
4        //Interface declaration
5        protected virtual simpleadder_if vif;
6
7        function new(string name, uvm_component parent);
8             super.new(name, parent);
9        endfunction: new
10
11       function void build_phase(uvm_phase phase);
12            super.build_phase(phase);
13       void'(uvm_resource_db#(virtual
```

```
   simpleadder_if)::read_by_name(.scope("ifs"), .name("simpleadder_if"),
14 .val(vif)));
15      endfunction: build_phase
16
17      task run_phase(uvm_phase phase);
18           //Our code here
19      endtask: run_phase
   endclass: simpleadder_driver
```

Code 5.1 – Driver component – *simpleadder_driver.sv*

The code might look complex already but what it's represented it's the usual code patterns from UVM. We are going to focus mainly on the *run_phase()* task which is where the behaviour of the driver will be stated. But before that, a simple explanation of the existing lines will be given:

- Line 1 derives a class named *simpleadder_driver* from the UVM class *uvm_driver*. The *#(simpleadder_transaction)* is a SystemVerilog parameter and it represents the data type that it will be retrieved from the sequencer.

- Line 2 refers to the UVM utilities macro explained on chapter 2.

- Lines 7 to 9 are the class constructor.

- Line 11 starts the build phase of the class, this phase is executed before the run phase.

- Line 13 gets the interface from the factory database. This is the same interface we instantiated earlier in the top block.

- Line 16 is the run phase, where the code of the driver will be executed.

Now that the driver class was explained, you might be wondering: "What exactly should I write in the run phase?"

Consulting the state machine from the chapter 1, we can see that the DUT waits for the signal *en_i* to be triggered before listening to the *ina* and *inb* inputs, so we need to emulate the states 0 and 1. Although we don't intend to sample the output of the DUT with the driver, we still need to respect it, which means, before we send another sequence, we need to wait for the DUT to output the result.

To sum up, in the run phase the following actions must be taken into account:

1. Get a sequence item

2. Control the *en_i* signal

3. Drive the sequence item to the bus

4. Wait a few cycles for a possible DUT response and tell the sequencer to send the next sequence item

The driver will end its operation the moment the sequencer stops sending transactions. This is done automatically by the UVM API, so the designer doesn't need to to worry with this kind of details.

In order to write the driver, it's easier to implement the code directly as a normal testbench and observe its behaviour through waveforms. As a result, in the next subchapter (chapter 5.1), the driver will first be implemented as a normal testbench and then we will reuse the code to implement the run phase (chapter 5.2).

Chapter 5.1 – Creating the driver as a normal testbench

For our normal testbench we will use regular Verilog code. We will need two things: generate the clock and idesginate an end for the simulation. A simulation of 30 clock cycles was defined for this testbench.

The code is represented in Code 5.2.

```
1    //Generates clock
2    initial begin
3         #20;
4        forever #20 clk = ! clk;
5    end
6
7    //Stops testbench after 30 clock cyles
8    always@(posedge clk)
9    begin
10       counter_finish = counter_finish + 1;
11
12       if(counter_finish == 30) $finish;
13   end
```

Code 5.2 – Clock generation for the normal testbench

The behaviour of the driver follows in Code 5.3.

```
1   //Driver
2   always@(posedge clk)
3   begin
4       //State 0: Drives the signal en_o
5       if(counter_drv==0)
6       begin
7           en_i = 1'b1;
8           state_drv = 1;
9       end
10
11      if(counter_drv==1)
12      begin
13          en_i = 1'b0;
14      end
15
16      case(state_drv)
17          //State 1: Transmits the two inputs ina and inb
18          1: begin
19              ina = tx_ina[1];
20              inb = tx_inb[1];
21
```

```
22              tx_ina = tx_ina << 1;
23              tx_inb = tx_inb << 1;
24
25              counter_drv = counter_drv + 1;
26              if(counter_drv==2) state_drv = 2;
27          end
28
29          //State 2: Waits for the DUT to respond
30          2: begin
31              ina = 1'b0;
32              inb = 1'b0;
33              counter_drv = counter_drv + 1;
34
35              //After the supposed response, the TB starts over
36              if(counter_drv==6)
37              begin
38                  counter_drv = 0;
39                  state_drv = 0;
40
41                  //Restores the values of ina and inb
42                  //to send again to the DUT
43                  tx_ina <= 2'b11;
44                  tx_inb = 2'b10;
45              end
46          end
47      endcase
48  end
```

Code 5.3 – Part of the driver

For this testbench, we are sending the values of *tx_ina* and *tx_inb* to the DUT, they are defined in the beginning of the testbench (you can see the complete code attached to this guide).

We are sending the same value multiple times to see how the driver behaves by sending consecutive transactions.

After the execution of the Makefile, a file named *simpleadder.dump* will be created by VCS. To see the waveforms of the simulation, we just need to open it with DVE.

The waveform for the driver is represented on Figure 5.1.

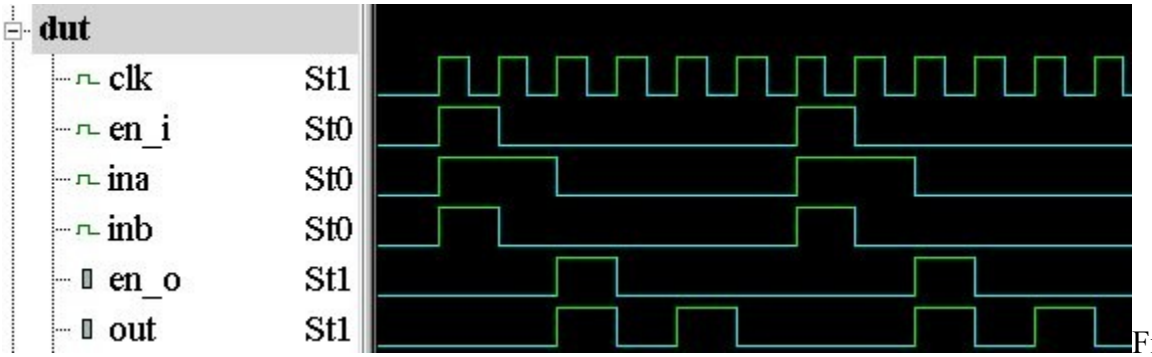


Figure 5.1 – Driver waveform

It's possible to see that the driver is working as expected: it drives the signal *en_i* on and off as well the DUT inputs *ina* and *inb* and it waits for a response of the DUT before sending the transaction again.

Chapter 5.2 – Implementing the UVM driver

After we have verified that our driver behaves as expected, we are ready to move the code into the run phase as seen in Code 5.4.

```
1   virtual task drive();
2       simpleadder_transaction sa_tx;
3       integer counter = 0, state = 0;
4       vif.sig_ina = 0'b0;
5       vif.sig_inb = 0'b0;
6       vif.sig_en_i = 1'b0;
7
8       forever begin
9           if(counter==0) begin
10              //Gets a transaction from the sequencer and
11              //stores it in the variable 'sa_tx'
12              seq_item_port.get_next_item(sa_tx);
13          end
14
15          @(posedge vif.sig_clock)
16          begin
17              if(counter==0) begin
18                  vif.sig_en_i = 1'b1;
19                  state = 1;
20              end
21
22              if(counter==1) begin
23                  vif.sig_en_i = 1'b0;
24              end
25
26              case(state)
27                  1: begin
28                      vif.sig_ina = sa_tx.ina[1];
29                      vif.sig_inb = sa_tx.inb[1];
30
31                      sa_tx.ina = sa_tx.ina &amp;lt;&amp;lt; 1;
32                      sa_tx.inb = sa_tx.inb &amp;lt;&amp;lt; 1;
33
34                      counter = counter + 1;
35                      if(counter==2) state = 2;
36                  end
37
38                  2: begin
39                      vif.sig_ina = 1'b0;
40                      vif.sig_inb = 1'b0;
41                      counter = counter + 1;
42
43                      if(counter==6) begin
44                          counter = 0;
45                          state = 0;
46
47                          //Informs the sequencer that the
48                          //current operation with
49                          //the transaction was finished
50                          seq_item_port.item_done();
```

```
51                      end
52                  end
53              endcase
54          end
55      end
56  endtask: drive
```

Code 5.4 – Task for the *run_phase()*

The ports of the DUT are acessed through the virtual interface with *vif.<signal>* as can be seen in lines 4 to 6.

Lines 12 and 50 use a special variable from UVM, the *seq_item_port* to communicate with the sequencer. The driver calls the method *get_next_item()* to get a new transaction and once the operation is finished with the current transaction, it calls the method *item_done()*. If the driver calls *get_next_item()* but the sequencer doesn't have any transactions left to transmit, the current task returns.

This variable is actually a UVM port and it connects to the export from the sequencer named *seq_item_export*. The connection is made by an upper class, in our case, the agent. Ports and exports are going to be further explained in chapter 6.0.1.

This concludes our driver, the full code for the driver can be found in the file *simpleadder_driver.sv*. In Figure 5.2, the state of the verification environment with the driver can be seen.

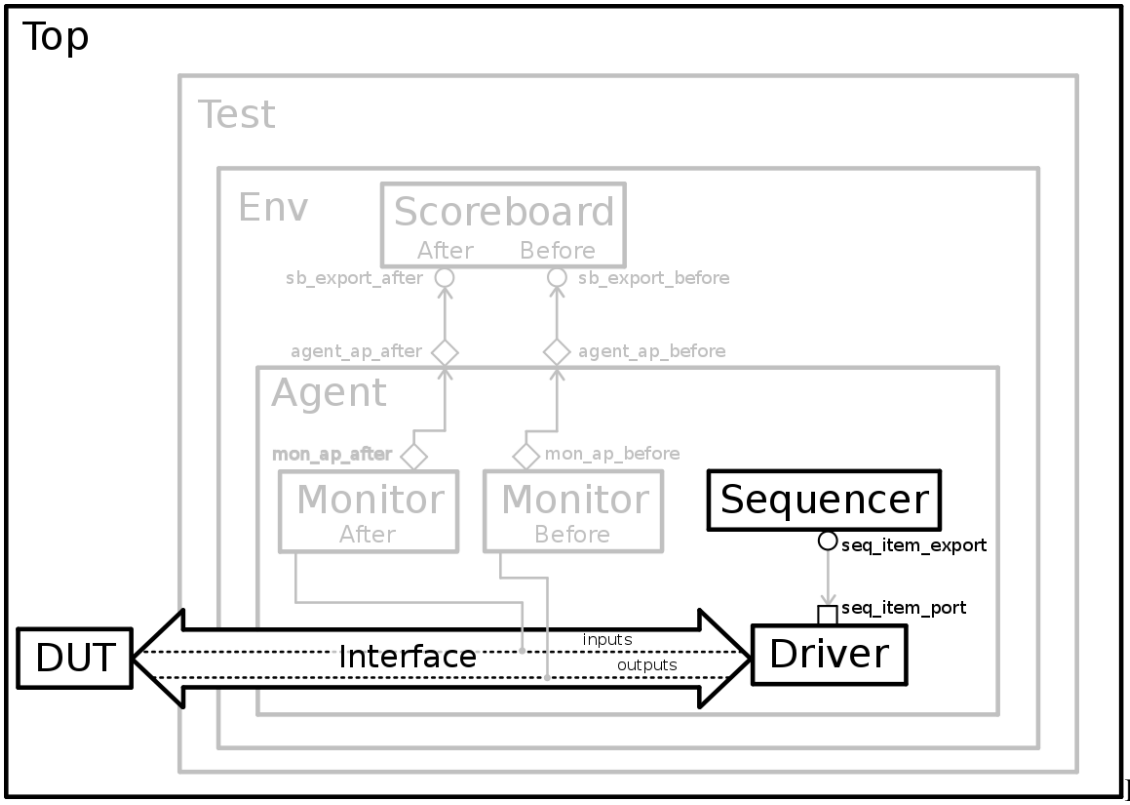


Figure 5.2 – State of the verification environment with the driver

The source of the driver can be accessed here:

Chapter 6 – Monitor

The monitor is a self-contained model that observes the communication of the DUT with the testbench. At most, it should observe the outputs of the design and, in case of not respecting the protocol's rules, the monitor must return an error.

The monitor is a passive component, it doesn't drive any signals into the DUT, its purpose is to extract signal information and translate it into meaningful information to be evaluated by other components. A verification environment isn't limited to just one monitor, it can have multiple of them.

The monitors should cover:

- The outputs of the DUT for protocol adherence
- The inputs of the DUT for functional coverage analysis

The approach we are going to follow for this verification plan is: sample both inputs, make a prediction of the expected result and compare it with the result of the DUT.

Consequently, we are going to create two different monitors:

- The first monitor, *monitor_before*, will look solely for the output of the device and it will pass the result to the scoreboard.

- The second monitor, *monitor_after*, will get both inputs and make a prediction of the expected result. The scoreboard will get this predicted result as well and make a comparison between the two values.

A portion of the code for both monitors can be seen in Code 6.1 and in Code 6.2.

```
class simpleadder_monitor_before extends uvm_monitor;
     `uvm_component_utils(simpleadder_monitor_before)

     uvm_analysis_port#(simpleadder_transaction) mon_ap_before;

     virtual simpleadder_if vif;

     function new(string name, uvm_component parent);
          super.new(name, parent);
     endfunction: new

     function void build_phase(uvm_phase phase);
          super.build_phase(phase);

          void'(uvm_resource_db#(virtual simpleadder_if)::read_by_name
(.scope("ifs"), .name("simpleadder_if"), .val(vif)));
          mon_ap_before = new(.name("mon_ap_before"), .parent(this));
     endfunction: build_phase

     task run_phase(uvm_phase phase);
          //Our code here
     endtask: run_phase
endclass: simpleadder_monitor_before
```

Code 6.1 – Code for *monitor_before*

```
class simpleadder_monitor_after extends uvm_monitor;
     `uvm_component_utils(simpleadder_monitor_after)

     uvm_analysis_port#(simpleadder_transaction) mon_ap_after;

     virtual simpleadder_if vif;

     simpleadder_transaction sa_tx_cg;

     covergroup simpleadder_cg;
          ina_cp:     coverpoint sa_tx_cg.ina;
          inb_cp:     coverpoint sa_tx_cg.inb;
          cross ina_cp, inb_cp;
     endgroup: simpleadder_cg

     function new(string name, uvm_component parent);
          super.new(name, parent);
          simpleadder_cg = new;
     endfunction: new

     function void build_phase(uvm_phase phase);
          super.build_phase(phase);

```

```
            void'(uvm_resource_db#(virtual
24 simpleadder_if)::read_by_name(.scope("ifs"), .name("simpleadder_if"),
25 .val(vif)));
26            mon_ap_after= new(.name("mon_ap_after"), .parent(this));
27      endfunction: build_phase
28
29      task run_phase(uvm_phase phase);
30            //Our code here
31      endtask: run_phase
   endclass: simpleadder_monitor_after
```

Code 6.2 – Code for *monitor_after*

The skeleton of both monitors is very similar to the driver, except for Lines 4. They represent one of the existing UVM communication ports. These ports allow different objects to pass transactions between them. In the section 6.0.1 you can consult a brief explanation of UVM ports.

The monitors will collect transactions from the virtual interface and use the analysis ports to send those transactions to the scoreboard. The code for the run phase can be designed the same way as for the driver but it was omitted in this section.

The full code for both monitors can be found in the file *simpleadder_monitor.sv*.

The the state of our verification environment after the monitors can be consulted in Figure 6.1.

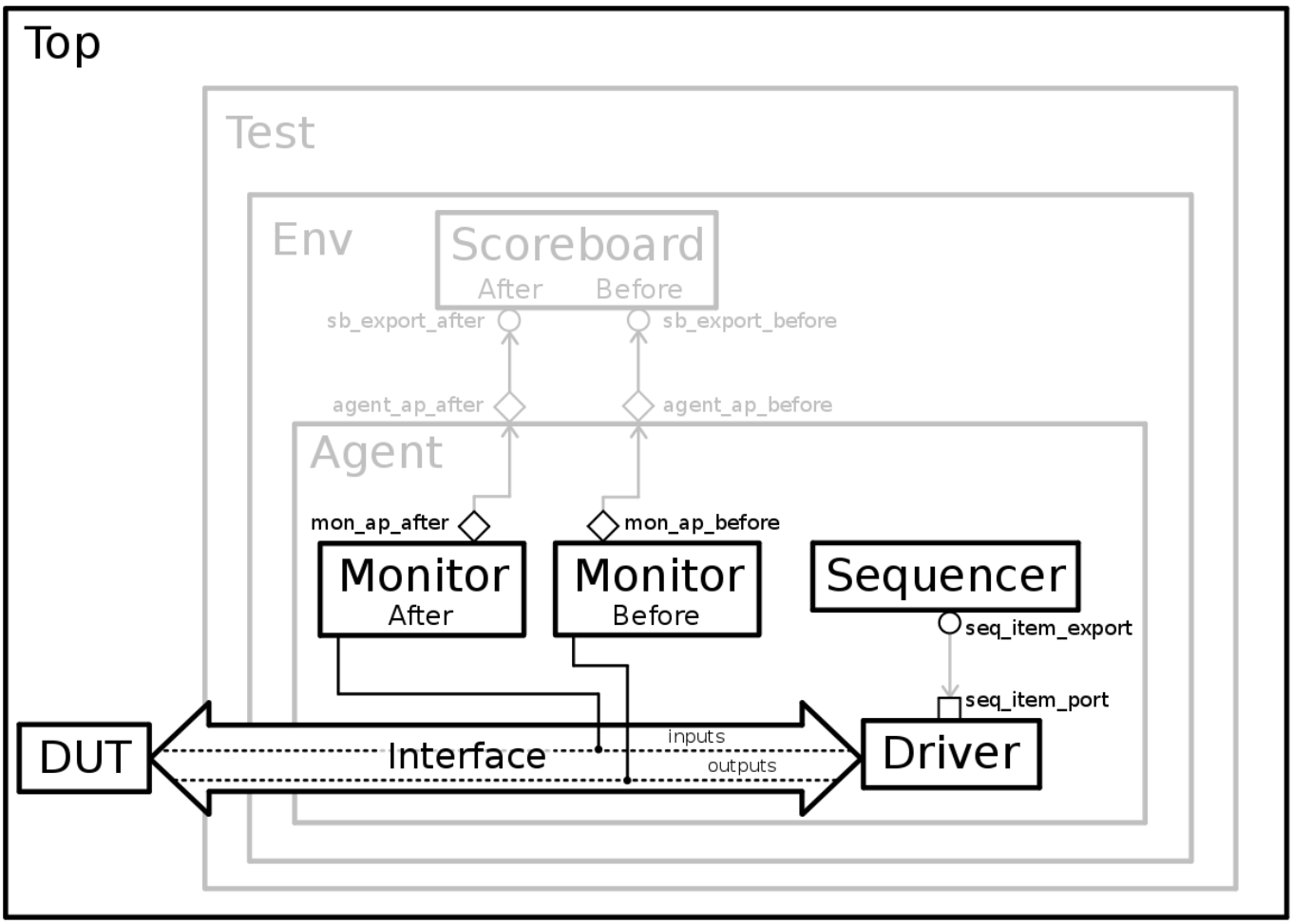


Figure 6.1 – State of the verification environment after the monitors

Chapter 6.0.1 – TLM ports

In chapter 4, it was mentioned that transactions are the most basic data transfer in a verification environment but another question arises: how do transactions are moved between components? We have already talked about TLM before when we were designing the driver. The way the driver gets transactions from the sequencer, it's the same way the scoreboard gets them from the monitors: through TLM.

TLM stands for *Transaction Level Modeling* and it's a high-level approach to modeling communication between digital systems. This approach is represented by two main aspects: *ports* and *exports*.

A TLM port defines a set of methods and functions to be used for a particular connection, while an export supplies the implementation of those methods. Ports and exports use transaction objects as arguments.

We can see a representation of a TLM connection in Figure 6.2.

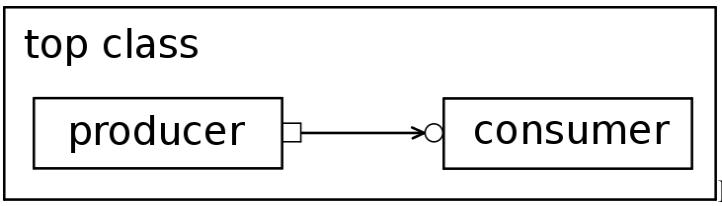


Figure 6.2 – Port-export communication

The communication is very easy to understand. The consumer implements a function that accepts a transaction as an argument and the producer calls that very function while passing the expected transaction as argument. The top block connects the producer to the consumer.

A sample code is provided in Table 6.1.

```
class topclass extends uvm_component;
    ...
    function void connect_phase(uvm_phase phase);
        ...
        producer.test_port.connect(consumer.test_export)
        ...
    endfunction;
endclass;
```

```
class producer extends uvm_component;
uvm_blocking_put_port#(test_transaction) test_port;

    ...
    task run();
        test_transaction t;
        test_port.testfunc(t);
    endtask
endclass
```

```
class consumer extends uvm_component;
uvm_blocking_put_imp#(test_transaction,
                      consumer) test_export;
    ...
    task testfunc(test_transaction t);
        //Code for testfunc() here...
    endtask

endclass
```

Table 6.1 – Sample code for ports and exports

The class *topclass* connects the producer's *test_port* to the consumer's *test_export* using the *connect()* method. Then, the producer executes the consumer's function *testfunc()* through *test_port*.

A particular characteristic of this kind of communication is that a port can only be connected to a single export. But there are cases when we might be interested in having a special port that can be plugged into several exports.

A third type of TLM port exists to cover these kind of cases: the analysis port.

An analysis port works exactly like a normal port but it can detect the number of exports that are connected to it and every time a required function is asked through this port, all other components whose exports are connected to an analysis port are going to be triggered.

In Figure 6.2 it's represented an analysis port communication.

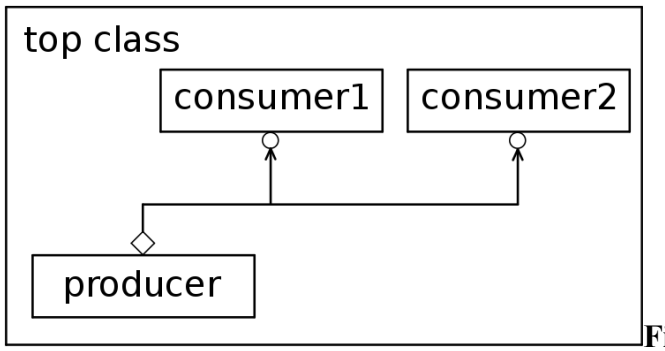


Figure 6.2 – Analysis port communication

The communication models mentioned here are part of Transaction Level Modeling 1.0. There is another variant, TLM 2.0, that works with sockets instead of ports, but they aren't going to be mentioned in this training guide.

A brief summary of these ports and exports can be seen in Table 6.2.

Symbol	Type	Port declaration
□	Port	uvm_blocking_put_port #(transaction) port_nam
○	Export	uvm_blocking_put_imp #(transaction, classname) expo
◇	Analysis Port	uvm_analysis_port #(transaction) analysis_port_na

Table 6.2 – Sum up of TLM-1.0 ports

For more information about TLM, you can consult:

- Accellera's UVM 1.1 User's Guide, page 11.

Chapter 7 – Agent

We have both monitors, the sequencer and the driver, so the next step is to connect them up. This is a job for the agent.

An agent doesn't require a run phase, there is no simulation code to be executed in this block but there will be a connect phase, besides of the build phase.

We will construct the monitors, the sequencer and the driver in the build phase. We will also need to create two analysis ports, these ports will act as proxies for the monitors to be connect to an external scoreboard through the agent's ports.

Note: We could have made the connection from the monitors directly to the scoreboard within the 'Env' class without passing by the agent's ports. There are cases when this is the best option, or other cases when it's not. It's always up to the designer to decide the best option.

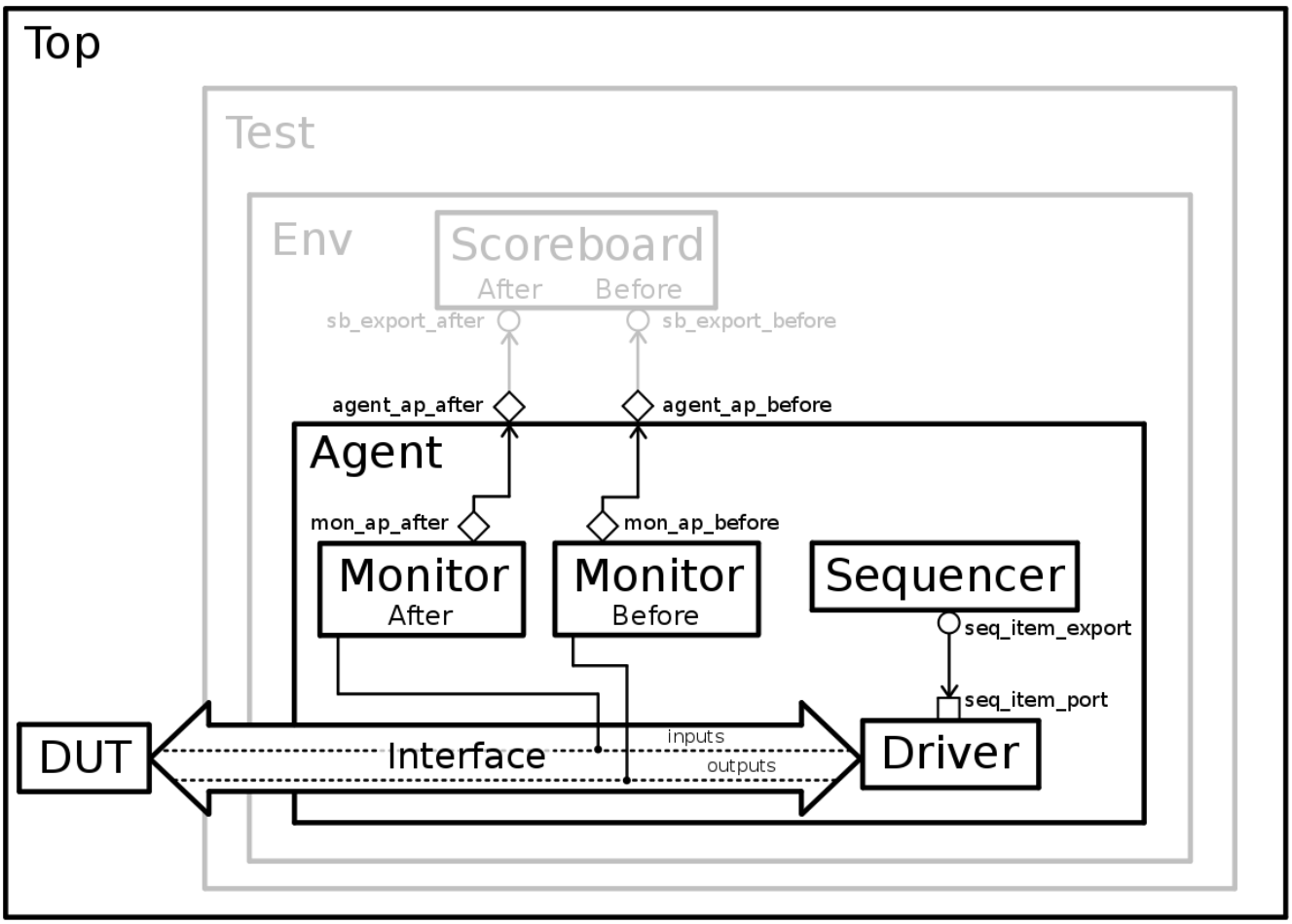
After we have constructed the components we need, we have to make the connections between them. Using the concepts learned in chapter 6.0.1 about TLM ports, we can connect each port to its destination.

A part of the code for the agent can be seen on Code 7.1.

```
   class simpleadder_agent extends uvm_agent;
        `uvm_component_utils(simpleadder_agent)
1
2       //Analysis ports to connect the monitors to the scoreboard
3       uvm_analysis_port#(simpleadder_transaction) agent_ap_before;
4       uvm_analysis_port#(simpleadder_transaction) agent_ap_after;
5
6       simpleadder_sequencer         sa_seqr;
7       simpleadder_driver         sa_drvr;
8       simpleadder_monitor_before     sa_mon_before;
9       simpleadder_monitor_after     sa_mon_after;
10
11      function new(string name, uvm_component parent);
12           super.new(name, parent);
13      endfunction: new
14
15      function void build_phase(uvm_phase phase);
16           super.build_phase(phase);
17
18           agent_ap_before     = new(.name("agent_ap_before"),
19 .parent(this));
20           agent_ap_after     = new(.name("agent_ap_after"),
21 .parent(this));
22
23           sa_seqr         = simpleadder_sequencer::type_id::create(...
24           sa_drvr         = simpleadder_driver::type_id::create(...
25           sa_mon_before     =
26 simpleadder_monitor_before::type_id::create(...
27           sa_mon_after     =
28 simpleadder_monitor_after::type_id::create(...
29      endfunction: build_phase
30
31      function void connect_phase(uvm_phase phase);
32           super.connect_phase(phase);
33           sa_drvr.seq_item_port.connect(sa_seqr.seq_item_export);
34           sa_mon_before.mon_ap_before.connect(agent_ap_before);
35           sa_mon_after.mon_ap_after.connect(agent_ap_after);
        endfunction: connect_phase
   endclass: simpleadder_agent
```

Code 7.1 – Code for the agent

In Figure 7.1, it's represented the current state of our testbench.



For more information about agents you can consult:

- Accellera's UVM 1.1 User's Guide, page 43
- Verification Academy's UVM Cookbook, page 42

The source of the agent component can be found here:

- simpleadder_agent.sv

Chapter 8 – Scoreboard

The scoreboard is a crucial element in a self-checking environment, it verifies the proper operation of a design at a functional level. This component is the most difficult one to write, it varies from project to project and from designer to designer.

In our case, we decided to make the prediction of the DUT functionality in the monitors and let the scoreboard compare the prediction with the DUT's response. But there are designers who prefer to leave the prediction to the scoreboard. So the functionality of the scoreboard is very subjective.

In the agent, we created two monitors, as a result, we will have to create two analysis exports in the scoreboard that are going to be used to retrieve transactions from both monitors. After

that, a method *compare()* is going to be executed in the run phase and compare both transactions. If they match, it means that the testbench and the DUT both agree in the functionality and it will return an "OK" message.

But we have a problem: we have two transaction streams coming from two monitors and we need to make sure they are synchronized. This could be done manually by writing appropriated *write()* functions but there is an easier and cleaner way of doing this: by using UVM FIFO.

These FIFO will work as it's represented in Figure 8.1.

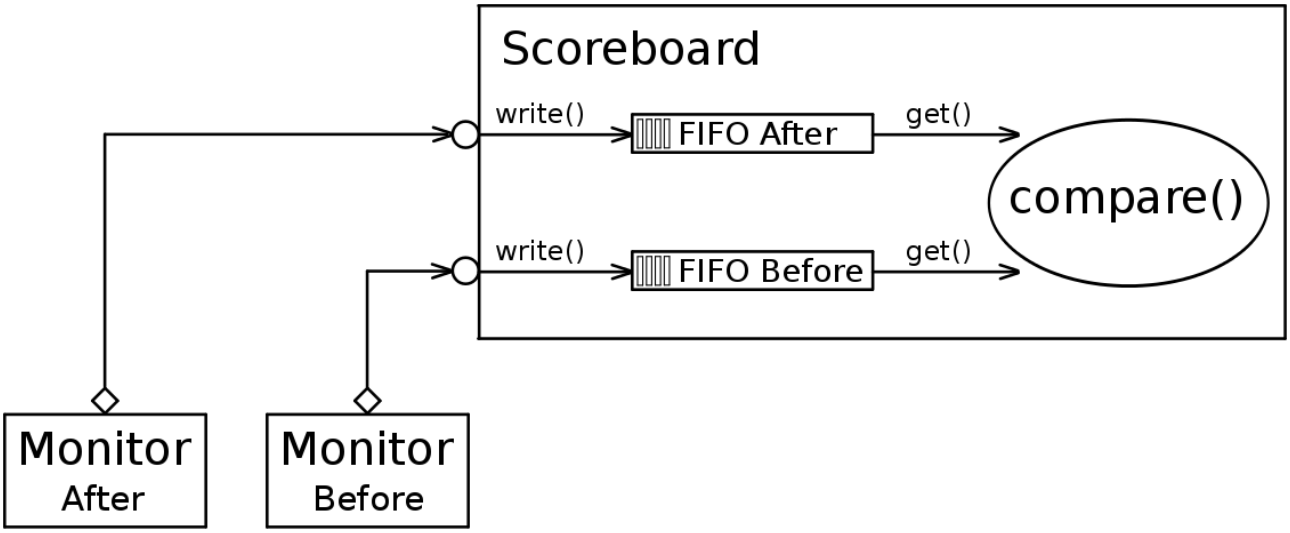


Figure 8.1 – Usage of FIFO in the scoreboard

The FIFO are instantiated similarly to ports/exports, with *uvm_tlm_analysis_fifo #(generic_transaction) generic_fifo* and they already implement the respective *write()* functions that are called from the monitors. To access their data we just execute the *get()* method from each FIFO.

The code from the scoreboard follows in Code 8.1.

```
1  class simpleadder_scoreboard extends uvm_scoreboard;
2       `uvm_component_utils(simpleadder_scoreboard)
3
4       uvm_analysis_export #(simpleadder_transaction) sb_export_before;
5       uvm_analysis_export #(simpleadder_transaction) sb_export_after;
6
7       uvm_tlm_analysis_fifo #(simpleadder_transaction) before_fifo;
8       uvm_tlm_analysis_fifo #(simpleadder_transaction) after_fifo;
9
10      simpleadder_transaction transaction_before;
11      simpleadder_transaction transaction_after;
12
13      function new(string name, uvm_component parent);
14           super.new(name, parent);
15           transaction_before    = new("transaction_before");
16           transaction_after     = new("transaction_after");
17      endfunction: new
18
```

```
19      function void build_phase(uvm_phase phase);
20           super.build_phase(phase);
21           sb_export_before    = new("sb_export_before", this);
22           sb_export_after       = new("sb_export_after", this);
23
24           before_fifo        = new("before_fifo", this);
25           after_fifo       = new("after_fifo", this);
26      endfunction: build_phase
27
28      function void connect_phase(uvm_phase phase);
29           sb_export_before.connect(before_fifo.analysis_export);
30           sb_export_after.connect(after_fifo.analysis_export);
31      endfunction: connect_phase
32
33      task run();
34           forever begin
35                before_fifo.get(transaction_before);
36                after_fifo.get(transaction_after);
37                compare();
38           end
39      endtask: run
40
41      virtual function void compare();
42           if(transaction_before.out == transaction_after.out) begin
43                `uvm_info("compare", {"Test: OK!"}, UVM_LOW);
44           end else begin
45                `uvm_info("compare", {"Test: Fail!"}, UVM_LOW);
46           end
47      endfunction: compare
48 endclass: simpleadder_scoreboard
```

Code 8.1 – Code for the scoreboard

In Figure 8.2, it's represented the current state of our testbench.

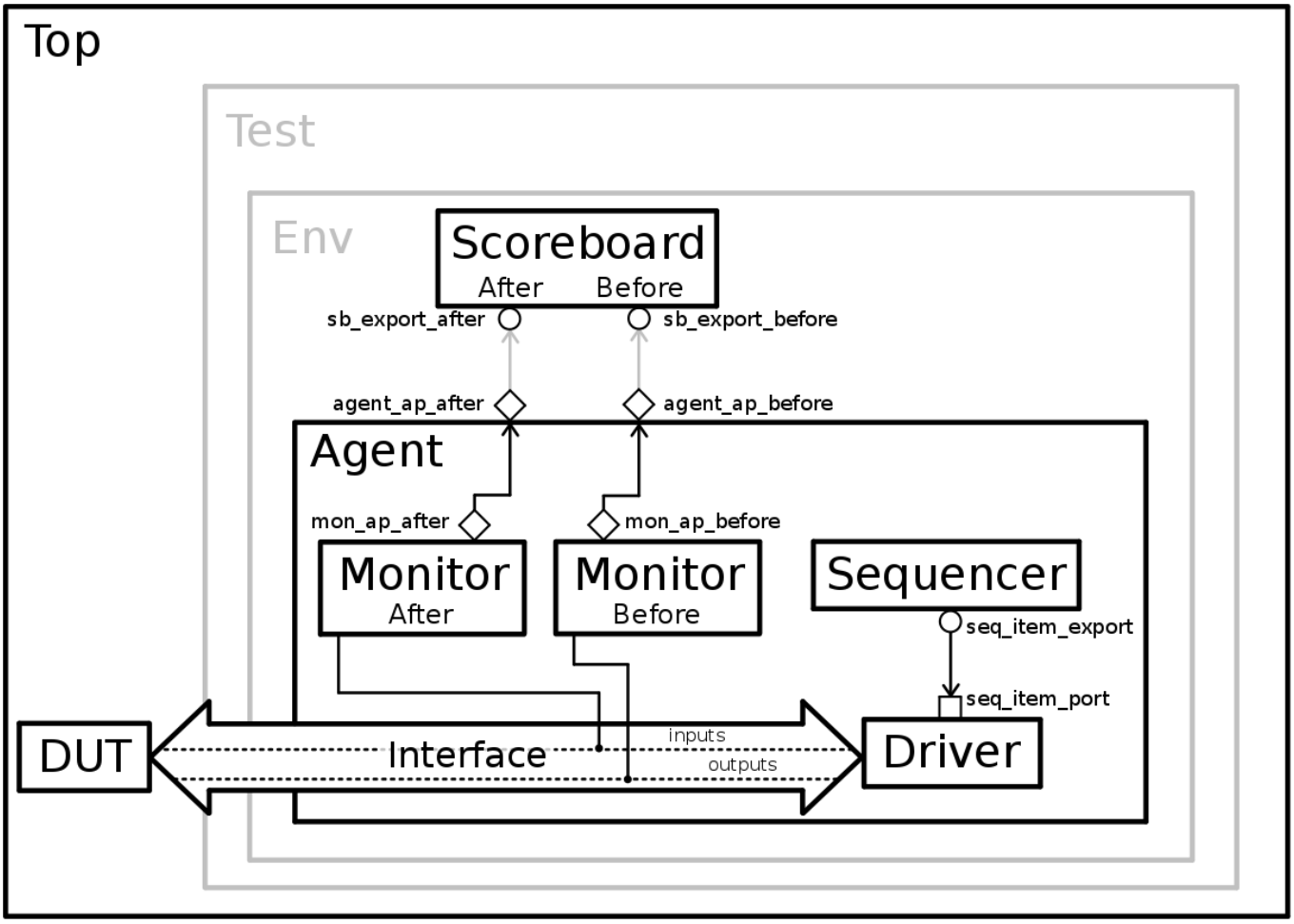


Figure 8.2 – State of the testbench after the scoreboard

For more information about scoreboards you can consult:

- Accellera's UVM 1.1 User's Guide, page 72
- Verification Academy's UVM Cookbook, pages 155 and 163
- Comprehensive Functional Verification: The Complete Industry Cycle, page 82

The code for the scoreboard can be consulted here:

- simpleadder_scoreboard.sv

Chapter 9 – Env

We are getting close to have a working testbench, there are two classes missing: the *env* and the *test*.

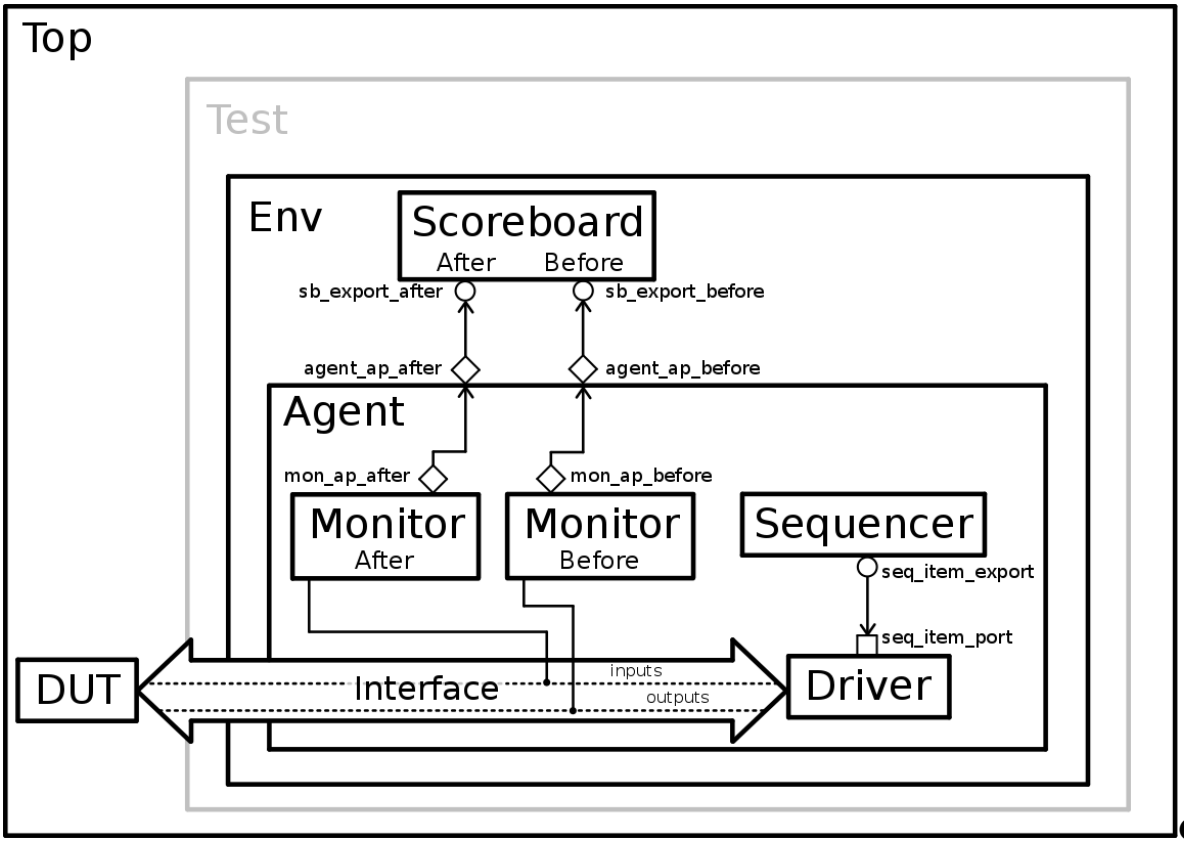
The *env* is a very simple class that instantiates the agent and the scoreboard and connects them together.

The source is represented in Code 9.1.

```
1  class simpleadder_env extends uvm_env;
2       `uvm_component_utils(simpleadder_env)
3
4       simpleadder_agent sa_agent;
5       simpleadder_scoreboard sa_sb;
6
7       function new(string name, uvm_component parent);
8            super.new(name, parent);
9       endfunction: new
10
11      function void build_phase(uvm_phase phase);
12           super.build_phase(phase);
13           sa_agent    = simpleadder_agent::type_id::create(...
14           sa_sb        = simpleadder_scoreboard::type_id::create(...
15      endfunction: build_phase
16
17      function void connect_phase(uvm_phase phase);
18           super.connect_phase(phase);
19           sa_agent.agent_ap_before.connect(sa_sb.sb_export_before);
20           sa_agent.agent_ap_after.connect(sa_sb.sb_export_after);
21      endfunction: connect_phase
22 endclass: simpleadder_env
```

Code 9.1 – Code for the *env*

In Figure 9.1, it's represented the current state of our testbench. There is only one component left now: the *test* class.



C ode 9.1 – State of the testbench after the env

The code for the *env* can be found here:

- simpleadder_env.sv

Chapter 10 – Test

At last, we need to create one more block: the test. This block will derive from the *uvm_test* class and it will have two purposes:

- Create the *env* block
- Connect the *sequencer* to the *sequence*

You might be wondering why are we connecting the sequencer and the sequence in this block, instead of the agent block or the sequence block.

The reason is very simple: by specifying in the test class which sequence will be going to be generated in the sequencer, we can easily change the kind of data is transmitted to the DUT without messing with the agent's or sequence's code.

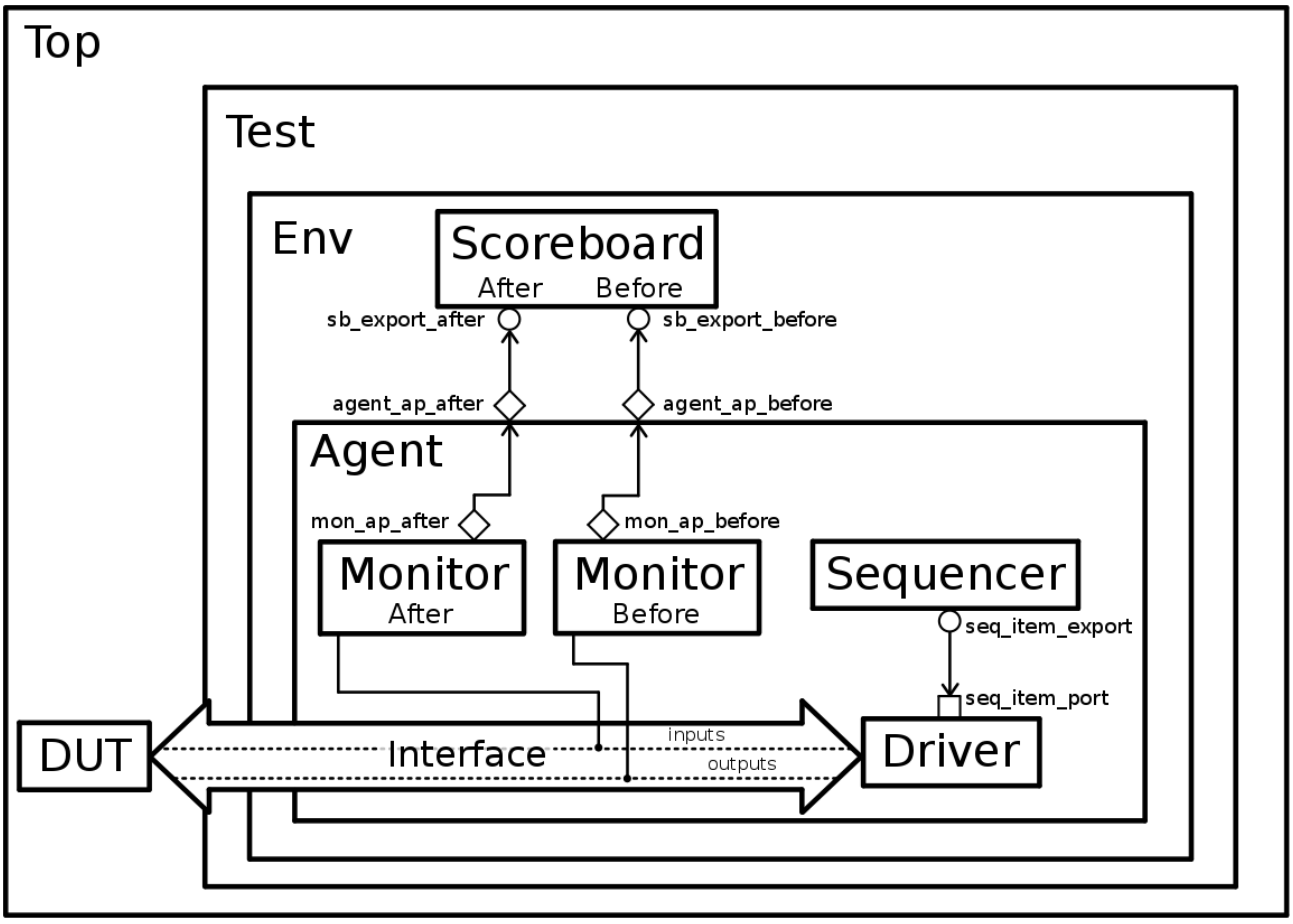
```
1  class simpleadder_test extends uvm_test;
2       `uvm_component_utils(simpleadder_test)
3
4       simpleadder_env sa_env;
```

```
5
6       function new(string name, uvm_component parent);
7            super.new(name, parent);
8       endfunction: new
9
10      function void build_phase(uvm_phase phase);
11           super.build_phase(phase);
12           sa_env = simpleadder_env::type_id::create(...
13      endfunction: build_phase
14
15      task run_phase(uvm_phase phase);
16           simpleadder_sequence sa_seq;
17
18           phase.raise_objection(.obj(this));
19                sa_seq = simpleadder_sequence::type_id::create(...
20                assert(sa_seq.randomize());
21                sa_seq.start(sa_env.sa_agent.sa_seqr);
22           phase.drop_objection(.obj(this));
23      endtask: run_phase
24 endclass: simpleadder_test
```

Code 10.1 – Code for the *test* class

Line 21 starts a sequencer in the desired sequence for this test.

In Figure 10.1, it's represented the current state of our testbench.



Code 10.1 – Final state of the testbench

Our testbench is finally ready, now it's time to execute it and check the results.

The source of the *test* component can be found here:

- simpleadder_test.sv

Chapter 11 – Running the simulation

To run the simulation, we simply execute the provided Makefile in the GitHub repository:

```
$ make -f Makefile.vcs
```

The testbench will generate random inputs and then those inputs will be sent to the DUT. The monitors will capture the data in the communication bus and make a prediction of the expected result. Finally the scoreboard will evaluate the functionality by matching the DUT's response with the prediction made by one of the monitors. If the DUT and the prediction match, an "OK" message will be outputted, otherwise, we will se a "Fail" message.

So, in the output of the simulation, we must find for the messages starting with *UVM_INFO* because the *compare()* method from the scoreboard is going to print a message using the macro *`uvm_info()* with the result of the test.

The result of the simulation can be seen below:

```
**********       IMPORTANT RELEASE NOTES        ************

You are using a version of the UVM library that has been compiled
with `UVM_NO_DEPRECATED undefined.
See http://www.eda.org/svdb/view.php?id=3313 for more details.

(Specify +UVM_NO_RELNOTES to turn off this notice)

UVM_INFO @ 0: reporter [RNTST] Running test simpleadder_test…
UVM_INFO simpleadder_scoreboard.sv(49) @ 70: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 130: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 190: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 250: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 310: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 370: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 430: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 490: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 550: uvm_test_top.sa_env.sa_sb [compare]
```

```
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 610: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 670: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 730: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 790: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO simpleadder_scoreboard.sv(49) @ 850: uvm_test_top.sa_env.sa_sb [compare]
Test: OK!
UVM_INFO ../uvm-src/uvm-1.1d/src/base/uvm_objection.svh(1268) @ 900: reporter
[TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase

— UVM Report Summary —

** Report counts by severity
UVM_INFO :   16
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[RNTST]     1
[TEST_DONE]     1
[compare]    14
$finish called from file "../uvm-src/uvm-1.1d/src/base/uvm_root.svh", line 430.
$finish at simulation time                 900
V C S   S i m u l a t i o n   R e p o r t
Time: 900 ns
CPU Time:      2.040 seconds;       Data structure size:   0.2Mb
```

This chapter concludes this beginner's guide. You can access all the code in this repository.

and the following is the code I changed a little.

```
`include "uvm_macros.svh"
```

```
interface simpleadder_if;
logic sig_clock;
logic sig_ina;
logic sig_inb;
logic sig_en_i;

logic sig_out;
logic sig_en_o;
endinterface: simpleadder_if
////////////////////////////////////////////////
////////////////////////////////////////////////
package simpleadder_pkg;
import uvm_pkg::*;

////////////////////////////////////////////////
//////////////////////////////////////////
class simpleadder_configuration extends uvm_object;
`uvm_object_utils(simpleadder_configuration)

function new(string name = “”);
super.new(name);
endfunction: new
endclass: simpleadder_configuration
////////////////////////////////////////////////
//////////////////////////////////////////
class simpleadder_transaction extends uvm_sequence_item;
rand bit[1:0] ina;
rand bit[1:0] inb;
bit[2:0] out;

function new(string name = “”);
super.new(name);
endfunction: new

`uvm_object_utils_begin(simpleadder_transaction)
`uvm_field_int(ina, UVM_ALL_ON)
`uvm_field_int(inb, UVM_ALL_ON)
`uvm_field_int(out, UVM_ALL_ON)
`uvm_object_utils_end
endclass: simpleadder_transaction

class simpleadder_sequence extends uvm_sequence#(simpleadder_transaction);
`uvm_object_utils(simpleadder_sequence)

function new(string name = “”);
super.new(name);
endfunction: new

task body();
simpleadder_transaction sa_tx;
```

```
repeat(5) begin
sa_tx = simpleadder_transaction::type_id::create(.name("sa_tx"), .contxt(get_full_name()));

start_item(sa_tx);
assert(sa_tx.randomize());
//`uvm_info("sa_sequence", sa_tx.sprint(), UVM_LOW);
finish_item(sa_tx);
end
endtask: body
endclass: simpleadder_sequence

typedef uvm_sequencer#(simpleadder_transaction) simpleadder_sequencer;

////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////
class simpleadder_driver extends uvm_driver#(simpleadder_transaction);
`uvm_component_utils(simpleadder_driver)

virtual simpleadder_if vif;

function new(string name, uvm_component parent);
super.new(name, parent);
endfunction: new

function void build_phase(uvm_phase phase);
super.build_phase(phase);

void'(uvm_resource_db#(virtual simpleadder_if)::read_by_name
(.scope("ifs"), .name("simpleadder_if"), .val(vif)));
endfunction: build_phase

task run_phase(uvm_phase phase);
drive();
endtask: run_phase

virtual task drive();
simpleadder_transaction sa_tx;
integer counter = 0, state = 0;
vif.sig_ina = 0'b0;
vif.sig_inb = 0'b0;
vif.sig_en_i = 1'b0;

forever begin
if(counter==0)
begin
seq_item_port.get_next_item(sa_tx);
//`uvm_info("sa_driver", sa_tx.sprint(), UVM_LOW);
end
```

```
@(posedge vif.sig_clock)
begin
if(counter==0)
begin
vif.sig_en_i = 1'b1;
state = 1;
end

if(counter==1)
begin
vif.sig_en_i = 1'b0;
end

case(state)
1: begin
vif.sig_ina = sa_tx.ina[1];
vif.sig_inb = sa_tx.inb[1];

sa_tx.ina = sa_tx.ina << 1;
sa_tx.inb = sa_tx.inb << 1;

counter = counter + 1;
if(counter==2) state = 2;
end

2: begin
vif.sig_ina = 1'b0;
vif.sig_inb = 1'b0;
counter = counter + 1;

if(counter==6)
begin
counter = 0;
state = 0;
seq_item_port.item_done();
end
end
endcase
end
end
endtask: drive
endclass: simpleadder_driver
//////////////////////////////////////////////////////////////
//////////////////////////////////////////////////////////
class simpleadder_monitor_before extends uvm_monitor;
`uvm_component_utils(simpleadder_monitor_before)

uvm_analysis_port#(simpleadder_transaction) mon_ap_before;

virtual simpleadder_if vif;
```

```
function new(string name, uvm_component parent);
super.new(name, parent);
endfunction: new

function void build_phase(uvm_phase phase);
super.build_phase(phase);

void'(uvm_resource_db#(virtual simpleadder_if)::read_by_name
(.scope("ifs"), .name("simpleadder_if"), .val(vif)));
mon_ap_before = new(.name("mon_ap_before"), .parent(this));
endfunction: build_phase

task run_phase(uvm_phase phase);
integer counter_mon = 0, state = 0;

simpleadder_transaction sa_tx;
sa_tx = simpleadder_transaction::type_id::create
(.name("sa_tx"), .contxt(get_full_name()));

forever begin
@(posedge vif.sig_clock)
begin
if(vif.sig_en_o==1'b1)
begin
state = 3;
end

if(state==3)
begin
sa_tx.out = sa_tx.out << 1;
sa_tx.out[0] = vif.sig_out;

counter_mon = counter_mon + 1;

if(counter_mon==3)
begin
state = 0;
counter_mon = 0;

//Send the transaction to the analysis port
mon_ap_before.write(sa_tx);
end
end
end
end
endtask: run_phase
endclass: simpleadder_monitor_before

class simpleadder_monitor_after extends uvm_monitor;
`uvm_component_utils(simpleadder_monitor_after)
```

```
uvm_analysis_port#(simpleadder_transaction) mon_ap_after;

virtual simpleadder_if vif;

simpleadder_transaction sa_tx;

//For coverage
simpleadder_transaction sa_tx_cg;

//Define coverpoints
covergroup simpleadder_cg;
ina_cp: coverpoint sa_tx_cg.ina;
inb_cp: coverpoint sa_tx_cg.inb;
cross ina_cp, inb_cp;
endgroup: simpleadder_cg

function new(string name, uvm_component parent);
super.new(name, parent);
simpleadder_cg = new;
endfunction: new

function void build_phase(uvm_phase phase);
super.build_phase(phase);

void'(uvm_resource_db#(virtual simpleadder_if)::read_by_name
(.scope("ifs"), .name("simpleadder_if"), .val(vif)));
mon_ap_after= new(.name("mon_ap_after"), .parent(this));
endfunction: build_phase

task run_phase(uvm_phase phase);
integer counter_mon = 0, state = 0;
sa_tx = simpleadder_transaction::type_id::create
(.name("sa_tx"), .contxt(get_full_name()));

forever begin
@(posedge vif.sig_clock)
begin
if(vif.sig_en_i==1'b1)
begin
state = 1;
sa_tx.ina = 2'b00;
sa_tx.inb = 2'b00;
sa_tx.out = 3'b000;
end

if(state==1)
begin
sa_tx.ina = sa_tx.ina << 1;
sa_tx.inb = sa_tx.inb << 1;
```

```
sa_tx.ina[0] = vif.sig_ina;
sa_tx.inb[0] = vif.sig_inb;

counter_mon = counter_mon + 1;

if(counter_mon==2)
begin
state = 0;
counter_mon = 0;

//Predict the result
predictor();
sa_tx_cg = sa_tx;

//Coverage
simpleadder_cg.sample();

//Send the transaction to the analysis port
mon_ap_after.write(sa_tx);
end
end
end
end
endtask: run_phase

virtual function void predictor();
sa_tx.out = sa_tx.ina + sa_tx.inb;
endfunction: predictor
endclass: simpleadder_monitor_after
//////////////////////////////////////////////////////////////////
//////////////////////////////////////////////////////////
class simpleadder_agent extends uvm_agent;
`uvm_component_utils(simpleadder_agent)

uvm_analysis_port#(simpleadder_transaction) agent_ap_before;
uvm_analysis_port#(simpleadder_transaction) agent_ap_after;

simpleadder_sequencer sa_seqr;
simpleadder_driver sa_drvr;
simpleadder_monitor_before sa_mon_before;
simpleadder_monitor_after sa_mon_after;

function new(string name, uvm_component parent);
super.new(name, parent);
endfunction: new

function void build_phase(uvm_phase phase);
super.build_phase(phase);
```

```
agent_ap_before = new(.name("agent_ap_before"), .parent(this));
agent_ap_after = new(.name("agent_ap_after"), .parent(this));

sa_seqr = simpleadder_sequencer::type_id::create(.name("sa_seqr"), .parent(this));
sa_drvr = simpleadder_driver::type_id::create(.name("sa_drvr"), .parent(this));
sa_mon_before = simpleadder_monitor_before::type_id::create(.name("sa_mon_before"),
.parent(this));
sa_mon_after = simpleadder_monitor_after::type_id::create(.name("sa_mon_after"),
.parent(this));
endfunction: build_phase

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);

sa_drvr.seq_item_port.connect(sa_seqr.seq_item_export);
sa_mon_before.mon_ap_before.connect(agent_ap_before);
sa_mon_after.mon_ap_after.connect(agent_ap_after);
endfunction: connect_phase
endclass: simpleadder_agent
//////////////////////////////////////////////////////////////
//////////////////////////////////////////////////////
// //
`uvm_analysis_imp_decl(_before)
`uvm_analysis_imp_decl(_after)

class simpleadder_scoreboard extends uvm_scoreboard;
`uvm_component_utils(simpleadder_scoreboard)

uvm_analysis_export #(simpleadder_transaction) sb_export_before;
uvm_analysis_export #(simpleadder_transaction) sb_export_after;

uvm_tlm_analysis_fifo #(simpleadder_transaction) before_fifo;
uvm_tlm_analysis_fifo #(simpleadder_transaction) after_fifo;

simpleadder_transaction transaction_before;
simpleadder_transaction transaction_after;

function new(string name, uvm_component parent);
super.new(name, parent);

transaction_before = new("transaction_before");
transaction_after = new("transaction_after");
endfunction: new

function void build_phase(uvm_phase phase);
super.build_phase(phase);

sb_export_before = new("sb_export_before", this);
sb_export_after = new("sb_export_after", this);
```

```
before_fifo = new("before_fifo", this);
after_fifo = new("after_fifo", this);
endfunction: build_phase

function void connect_phase(uvm_phase phase);
sb_export_before.connect(before_fifo.analysis_export);
sb_export_after.connect(after_fifo.analysis_export);
endfunction: connect_phase

task run();
forever begin
before_fifo.get(transaction_before);
after_fifo.get(transaction_after);

compare();
end
endtask: run

virtual function void compare();
if(transaction_before.out == transaction_after.out) begin
`uvm_info("compare", {"Test: OK!"}, UVM_LOW);
end else begin
`uvm_info("compare", {"Test: Fail!"}, UVM_LOW);
end
endfunction: compare
endclass: simpleadder_scoreboard
//////////////////////////////////////////////////////////////
class simpleadder_env extends uvm_env;
`uvm_component_utils(simpleadder_env)

simpleadder_agent sa_agent;
simpleadder_scoreboard sa_sb;

function new(string name, uvm_component parent);
super.new(name, parent);
endfunction: new

function void build_phase(uvm_phase phase);
super.build_phase(phase);
sa_agent = simpleadder_agent::type_id::create(.name("sa_agent"), .parent(this));
sa_sb = simpleadder_scoreboard::type_id::create(.name("sa_sb"), .parent(this));
endfunction: build_phase

function void connect_phase(uvm_phase phase);
super.connect_phase(phase);
sa_agent.agent_ap_before.connect(sa_sb.sb_export_before);
sa_agent.agent_ap_after.connect(sa_sb.sb_export_after);
endfunction: connect_phase
endclass: simpleadder_env
//////////////////////////////////////////////////////////////
```

```
class simpleadder_test extends uvm_test;
`uvm_component_utils(simpleadder_test)

simpleadder_env sa_env;

function new(string name, uvm_component parent);
super.new(name, parent);
endfunction: new

function void build_phase(uvm_phase phase);
super.build_phase(phase);
sa_env = simpleadder_env::type_id::create(.name("sa_env"), .parent(this));
endfunction: build_phase

task run_phase(uvm_phase phase);
simpleadder_sequence sa_seq;

phase.raise_objection(.obj(this));
sa_seq = simpleadder_sequence::type_id::create(.name("sa_seq"), .contxt(get_full_name()));
assert(sa_seq.randomize());
sa_seq.start(sa_env.sa_agent.sa_seqr);
phase.drop_objection(.obj(this));
endtask: run_phase
endclass: simpleadder_test
endpackage

//////////////////////////////////////////////////////
//
module simpleadder(input wire clk,

input wire en_i,
input wire ina,
input wire inb,

output reg en_o,
output reg out);

integer counter, state;
reg[1:0] temp_a, temp_b;
reg[2:0] temp_out;

//Init
initial begin
counter = 0;
temp_a = 2'b00;
temp_b = 2'b00;
temp_out = 3'b000;
out = 0;
```

```
en_o <= 0;
state = 0;
end

always@(posedge clk)
begin
//State 0: Wait for en_i
if(en_i==1'b1)
begin
state = 1;
end

case(state)
//State 1: Start reading inputs
1: begin
temp_a = temp_a << 1;
temp_a = temp_a | ina;

temp_b = temp_b << 1;
temp_b = temp_b | inb;

counter = counter + 1;

//After 2 bits, do the operation an move to the next state
if(counter==2) begin
temp_out = temp_a + temp_b;

state = 2;
end
end

//State 2: Enable en_o and sends result to the output
2: begin
out <= temp_out[2];
temp_out = temp_out << 1;

counter = counter + 1;

if(counter==3) en_o <= 1'b1;

if(counter==4) en_o <= 1'b0;

if(counter==6) begin
counter = 0;
out <= 1'b0;
state = 0;
end
end
endcase
end
```

```
endmodule
////////////////////////////////////////////////////
//
module simpleadder_tb_top;
import uvm_pkg::*;
import simpleadder_pkg::*;

//Interface declaration
simpleadder_if vif();

//Connects the Interface to the DUT
simpleadder dut(vif.sig_clock,
vif.sig_en_i,
vif.sig_ina,
vif.sig_inb,
vif.sig_en_o,
vif.sig_out);

initial begin
//Registers the Interface in the configuration block so that other
//blocks can use it
uvm_resource_db#(virtual simpleadder_if)::set
(.scope("ifs"), .name("simpleadder_if"), .val(vif));

//Executes the test
run_test();
end

//Variable initialization
initial begin
vif.sig_clock <= 1'b1;
end

//Clock generation
always
#5 vif.sig_clock = ~vif.sig_clock;
endmodule
////////////////////////////////////////////////////////////
```